

一. 程序阅读

1. 阅读以下 4 段关于内存的 C 代码，请回答运行其中的 Test 函数会有什么样的结果？

1.1

```
1.1
void GetMemory(char *p)
{
    p = (char *)malloc(100);
}

void Test(void)
{
    char *str = NULL;
    GetMemory(str);
    strcpy(str, "hello world");
    printf(str);
}
```

答案：运行结果是 strcpy 会出现段错误，因为操作了空指针 str，形参 p 的空间改变并不影响到实参 str 的空间

分析：本题考查的是函数形参和实参的区别和应用

1.2

```
1.2
char *GetMemory(void)
{
    char p[] = "hello world";
    return p;
}

void Test(void)
{
    char *str = NULL;
    str = GetMemory();
    printf(str);
}
```

答案：输入的结果为 NULL 或乱码（根据运行系统决定），p 的数据是存放在栈空间，程序运行完空间自动释放，所以 return 回的东西只是 p 的指针变量

分析：本题考察的是栈空间和堆空间的区别

1.3

```
1.3
void GetMemory(char **p, int num)
{
    *p = (char *)malloc(num);
}

void Test(void)
{
    char *str = NULL;

    GetMemory(&str, 100);
    strcpy(str, "hello");
    printf(str);
}
```

答案：打印出“hello”，通过&str 将 str 指针本身变量值传递过去了，所以 malloc 申请的空间首地址就是 str

分析：本题考察的是二级指针的应用

1.4

```
1.4
void Test(void)
{
    char *str = (char *) malloc(100);
    strcpy(str, "hello");
    free(str);
    if(str != NULL)
    {
        strcpy(str, "world");
        printf(str);
    }
}
```

分析：内存申请和释放的应用

答案：会打印出“world”，但也会出现段错误，因为申请的空间已经释放了，strcpy 访问了非法区域。

二. 程序实现

2.1

2.1 已知类 String 的原型为:

```
class String
{
public:
    String(const char *str = NULL); // 普通构造函数
    String(const String &other);    // 拷贝构造函数
    ~String(void);                 // 析构函数
    String & operate =(const String &other); // 赋值函数
    int Find(const char *str);      // 字符串查找函数
private:
    char    *m_data;               // 用于保存字符串
};
```

请编写 String 的上述 5 个函数的实现。

答案:

```
3 String::String(const char *str)
4 {
5     int length = strlen(str);
6     m_data = new char[length+1];
7     strcpy(m_data, str);
8 }
9 String::String(const String &other)
10 {
11     int length = strlen(other.m_data);
12     m_data = new char[length+1];
13     strcpy(m_data, other.m_data);
14 }
15 String::~String(void)
16 {
17     delete[] m_data;
18 }
19 String& String::operator =(const String &other)
20 {
21     if(*this == other) {
22         return other;
23     } else {
24         delete[] m_data;
25         int length = strlen(other.m_data);
26         m_data = new char[length+1];
27         strcpy(m_data, other.m_data);
28         return *this;
29     }
30 }
31 int String::Find(const char *str)
32 {
33     int length = strlen(str);
34     int i = 0;
35     while (*m_data != '\0') {
36         if(strcmp(m_data, str, length) == 0){
37             break;
38             return i+1;
39         }
40         ++m_data;
41         ++i;
42     }
43     return 0;
44 }
45 }
46
```

分析: 综合查考一个类的基本操作, 构造, 析构, 拷贝构造, 重载, 类函数

2.2

2.2

写一个函数找出一个整数数组中，第二大的数，函数定义如下，请完成函数的实现。

```
int find_sec_max( int data[] , int count)
{
}
```

答案：冒泡排序第一层循环两次就可以了

```
10 int find_sec_mac(int data[],int count)
11 {
12     int i=0,j=0,number=0;
13     for( ; i < 2 ;++i) {
14         for( ; j < count-1-i ; ++j) {
15             if(data[j] > data[j+1]) {
16                 number = data[j];
17                 data[j] = data[j+1];
18                 data[j+1] = number;
19             }
20         }
21     }
22     return data[count-2];
23 }
24
```

分析：考察数组排序和算法

三. 数据结构

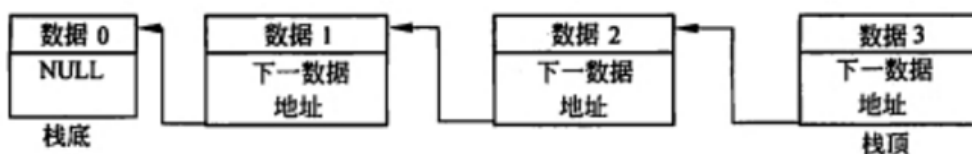
1. 栈(stack)结构是计算机语言实现中的一种重要数据结构。对于任意栈进行插入和删除操作的一端称为栈顶(stack top)，而另一端称为栈底(stack bottom)。栈的基本操作包括：创建栈(NewStack)、判断栈是否为空(IsEmpty)、判断栈是否已满(IsFull)、获取栈顶数据(Top)、压栈/入栈(Push)、弹栈/出栈(Pop)。当设计栈的存储结构时，可以采取多种方式。其中，采用链式存储结构实现的栈中各数据项不必连续存储。

(1) 请画出链式存储的栈的结构示意图

(2) 请用伪码或文字描述出上述栈的基本操作的实现

答案：

(1).



(2).

```
//链栈的类型定义
typedef struct node
{
    datatype data;          /*数据域*/
    struct node * next;     /*指针域*/
}LinkStack;                /*链栈结点类型*/
//创建新栈
LinkStack* NewStack()
{
    LinkStack *p = (LinkStack*)malloc(sizeof(LinkStack));
    p->next = NULL;
    return p;
}
//判别空栈
int StackEmpty(LinkStack *top)
{
    return (top?0:1);
}
//判断满栈
int StackFull(LinkStack *bottom)
{
    return (bottom?0:1);
}
//取栈顶元素
datatype GetTop(LinkStack *top)
{
    if(!top) {
        printf("/n链表是空的!");
        return 0;
    }
    return top->data;
}
//入栈
LinkStack *Push(LinkStack *top,datatype x)
{
    LinkStack *p;
    p=(LinkStack *)malloc(sizeof(LinkStack)); //分配空间
    p->data=x;          /*设置新结点的值*/
    p->next=top;        /*将新元素插入栈中*/
    top=p;              /*将新元素设为栈顶*/
    return top;
}
//出栈
LinkStack *Pop(LinkStack *top)
{
    LinkStack *p;
    if(!top) {
        printf("/n链栈是空的!");
        return NULL;
    } //判断是否为空栈
    p=top; //指向被删除的栈顶
    top=top->next; //修改栈顶指针
    free(p);
    return top;
}
```

分析：考察数据结构栈的基本知识

四.Windows 编程

4.1

4.1 线程知识

请列举与“线程”的创建、暂停运行、恢复运行、终止等有关的 windows API 函数，并进行简要介绍。

答案：首先，是创建一个线程，MSDN 提供了一个 CreateThread 的 API，用于创建一个线程，其声明如下：

```

HANDLE CreateThread(
    LPSECURITY_ATTRIBUTES lpThreadAttributes, // 安全属性, 一般为NULL
    DWORD dwStackSize, // 初始的栈大小, 给NULL即可 (系统会设置默认值)
    LPTHREAD_START_ROUTINE lpStartAddress, // 线程函数地址
    LPVOID lpParameter, // 线程参数, 传递给线程函数的参数
    DWORD dwCreationFlags, // 创建的操作, 如挂起等操作, 为NULL则立即运行
    LPDWORD lpThreadId // 线程的ID, 一般不需要, 给NULL
);

```

第二个是线程函数的写法, 格式不能修改, 名字可以改, 其声明如下:

```

DWORD WINAPI ThreadProc( LPVOID lpParameter /*上一个函数的lpParameter*/ );

```

第三个是 ResumeThread, 用于恢复线程运行, 其声明如下:

```

DWORD ResumeThread( HANDLE hThread // 线程句柄 );

```

第四个是 SuspendThread, 用于挂起线程, 不让他运行, 其声明如下:

```

DWORD SuspendThread( HANDLE hThread // 线程句柄 );

```

第五个是 TerminateThread, 用于杀死一个线程, 其声明如下:

```

BOOL TerminateThread(
    HANDLE hThread, // 线程句柄
    DWORD dwExitCode // 退出码, 一般用不到, 随便给个即可
);

```

4.2

4.2 消息知识

请列举与“消息”的发送、获取、分发、检测、处理等有关的 windows API 函数, 并进行简要介绍。

答案:

消息发送分两种

发送 (Send) 消息-直接发送给指定的窗口, 并等候结果.

投递 (Post) 消息-发送到消息队列当中, 立刻返回, 由消息循环处理.

PostMessage 和 SendMessage

PostMessage 产生队列消息, 由于发送后不等候消息处理结果, 所以不能确定消息是否被处理成功.

SendMessage 产生非队列消息, 可以确定消息是否成功.

消息循环

1.1 GetMessage 从队列中获取消息, 判断是否是 WM_QUIT 消息, 如果发现是 WM_QUIT 消息, 消息循环结束, 否则继续下一步.

1.2 TranslateMessage 翻译按键消息, 如果发现有按键消息, 产生字符消息放入消息队列, 继续下一步

1.3 DispatchMessage 找到消息所发窗口的处理函数, 处理消息. 处理完成后, 返回 1.1.

GetMesssage 和 PeekMessage

2.1 从线程消息队列中获取消息, 如果找到消息, 就返回消息, 进行消息处理. 如果未找到消息, 执行 2.2

2.2 查找系统消息队列. 通过向系统消息队列查询, 如果找到消息, 获取消息并返回,

进行消息处理. 如果未找到消息, 执行 2.3

2.3 检查窗口需要重新绘制的范围, 如果发现存在重新绘制的范围, 会产生 WM_PAINT 消息. 然后进行消息处理, 如果未找, 执行 2.4

2.4 检查 WM_TIMER 定时器消息, 如果发现存在已经到时的定时器, 会产生 WM_TIMER 消息. 进行消息处理. 如果未找, 执行 2.5

2.5 执行内存管理工作.

2.6 根据函数不同, 处理结果不同:

GetMesssage - 阻塞, 等候下一条消息

PeekMessage - 让出控制权, 交给后面的代码执行.

4.3

4.3 通讯知识

请分别以示意图描述客户端和服务器的 TCP 通讯流程, 并注明相关的 Socket API 及其简要说明。

答案:



1. WSAStartup 初始化 Ws2_32.dll 的函数

WSAStartup 函数用于初始化供进程调用的 Winsock 相关的 dll。

2. WSACleanup 释放 Ws2_32.dll 的 1 函数

该函数释放对 Winsock 链接库的调用。

3. socket 创建 socket 的函数

socket 函数将创建指定传输服务的 socket。

4. bind 服务端将 socket 与地址关联

5. listen 服务端网络监听

6. accept 服务端 connect 接收

7. connect 客户端请求服务端连接

8. send、recv 发送接收数据

9. closesocket 关闭 socket

10. shutdown 禁止在 socket 收发数据

11. getsockname 获取本地 IP 和 PORT

12. getpeername 获取对端 IP 和 PORT

五. 数据库

5.1

5.1

表: test1

ID	Varchar(6)	主键
NAME	Varchar(15)	

表: Test2

Use_ID	Varchar(6)	主键关联TEST1的主键的ID
PASSword	Varchar(10)	

用一条 SQL 语句查询 ID, NAME, PASSWORD, 并且当 PASSWORD 为空时, 用 “NO PASSWORD” 代替。

答案:

```
select ID,NAME,ifnull(PASSword,' NO PASSWORD' ) as PASSWORD from test1  
inner join test2 on test1.ID = test2.Use_ID;
```

分析: 考察多表联查、常用函数、别名

5.2

5.2

表: info

useID	useDate(交费时间)	Uselost(交费)
John	2006年2月	500
Tom	2006年3月	400
John	2006年3月	450

用户每月都要交费, 上表是各个用户的交费情况, 表中记录仅为部分。

要求用一条sql语句查询useID, useDate, Uselost, 并且符合Uselost大于400的条件, 并且按照useID升序, useDate降序排列, 并且只输出前20条记录。

答案:

```
Select useID,useDate,Userlost from info where Uselost > 400 limit 20  
order by useID asc , useDate desc;
```

分析: 考察 order by 子句、运算符、limit 子句

5.3

5.3

表：同5.2的info

要求用一条SQL语句查询出每个用户USEID，（交费次数）NUMBER，所交的个人总费用COST。输出结果如下表形式。

useID	number	Cost
John	2	
Tom	1	950
.....		400
		

答案：

Select useID, count(useID) as number, sum(Uselost) as Cost from info group by useID;

分析：selec 子句、order by 子句、常用函数、group by 子句、as

5.4

5.4

表：同5.2的info

要求用SQL语句查询出每个用户useID，每次交费时间useDate、金额uselost及本次交费后累计交费总额Cost。输出结果如下表形式。

useID	useDate (交费时间)	Uselost (交费)	Cost (累计交费)
John	2006年2月	500	500
Tom	2006年3月	400	400
John	2006年3月	450	950
.....			

答案：

Select useID, useDate, Uselost from info union select sum(Uselost) from info where ID = (where distinct useID from info);

分析：考察 select 子句、union 子句、子查询、常用函数、Distinct 关键字

5.5

5.5

如何用 sql 语句高效删除一个数据量巨大的表中的所有记录？

答案：

Delete from tables_name;

分析：清空表、delete 语句

5.6

5.6

如何取得指定月份的最后一天的日期？

答案:

```
Select date( 'now' , ' start of month' , ' +n month' , ' -1 day' );
```

分析: sqlite 时间与日期

5.7

5.7

请用 sql 语句查询出数据库 test 中所有的用户创建的（非系统）表。

答案: .tables

分析: sqlite 命令

5.8

5.8

请论述临时表的概念和作用。

答案:

临时表

1. 创建表时用 TEMPORARY 关键字修饰 table, 则所创建的表就是临时表了
 2. 本地临时表仅在当前会话中可见, 在会话结束时, 自动清除, 数据存放在 tempdb 中,
 3. 存储过程中的临时表, 在存储过程运行完成时将自动除去在存储过程中创建的本地临时表,
 4. 用法和实体表是一样的, 可以存放临时数据, 可以进行 select/update/delete
- 临时表用法之一: 因为 Sqlite 无法删除列, 当需要对原有表的列级别数据进行删除或者调整时就需要临时表来作为中转站了。

5.9

5.9

请论述索引的概念和作用。

答案:

索引 Index

索引 (Index) 是一种特殊的查找表, 数据库搜索引擎用来加快数据检索。类似与一本书有目录一样, 索引就是一些大容量的表的目录, 可以让 select 的速度更快, 更高效。多用于一些大型服务器的数据管理。